



POLITECNICO DI TORINO

MASTER'S DEGREE IN CYBERSECURITY ENGINEERING

Design and Implementation of an eBPF-based Security Monitoring Tool

Supervisor:

Prof. Riccardo Sisto

Candidate:

Simone Romano

Company Tutors:

Dr. David Soldani

Dr. Vincenzo Cosi

Academic Year 2025-2026

This thesis presents the design and implementation of a security monitoring tool that collects and analyzes network connection data directly from the operating system kernel using eBPF. By leveraging kernel-level instrumentation, the system captures detailed information about network activity with high accuracy and minimal overhead, overcoming the limitations of traditional user-space monitoring approaches.

The proposed tool adopts a modular and efficient architecture to ensure scalability and real-time data collection. The gathered data is structured to support security analysis, enabling the detection of anomalous behaviors such as suspicious outbound connections, unauthorized communication patterns, and potential data exfiltration attempts.

This work demonstrates how kernel-derived network telemetry, combined with eBPF technology, can enhance system visibility and provide a solid foundation for advanced network security monitoring and threat detection.

Acknowledgements

Only in the printed version

Contents

Acronyms	1
1 Introduction	2
1.1 Context and Motivation	2
1.2 Thesis Objectives	3
1.3 Scope of the Work	4
1.4 Thesis Structure	4
A Appendix A	6
B Appendix B	7

List of Figures

List of Tables

Acronyms

eBPF	Extended Berkeley Packet Filter
OS	Operating System
NIDS	Network Intrusion Detection System
JIT	Just-in-Time
API	Application Programming Interface
MVP	Minimum Viable Product
MITRE	MITRE Corporation

Chapter 1

Introduction

1.1 Context and Motivation

The extended Berkeley Packet Filter (eBPF) is a new programmability mechanism that changes the way we can observe and interact with the Linux kernel. Originally it was designed as a packet filtering mechanism, but now the paradigm has evolved to a more general purpose, addressing the need for observability and security in the Linux kernel.

eBPF introduces a tradeoff in kernel development: on one hand, it allows developers to extend the kernel with new features without the need to modify the kernel source code; but on the other hand, this widely adopted Linux kernel technology presents a new challenge for security, because it allows developers to run custom code in kernel space, which is a privileged environment.

When we use this mechanism to load custom programs in the kernel, we need to be sure that the code is safe and does not compromise the security of the system. eBPF addresses this problem by introducing a sophisticated verifier framework. This framework is able to analyze the code before it is loaded in the kernel, and to reject any code that does not respect the security and performance constraints. It permits programs running in the user space to submit eBPF bytecode to the kernel, which is then verified.

Once the programs are verified, they are executed in the kernel space, achieving high performance and low overhead through Just-in-Time (JIT) compilation, while maintaining or even enhancing the security of the kernel.

eBPF allows attaching eBPF programs to specific kernel hooks, enabling the observation and collection of events in the kernel space, without the need to modify the kernel source code.

This capability enables a new approach to security monitoring, allowing us to observe security-relevant events in a reliable and structured way, collecting them from different kernel hooks, transporting them to the user space, decoding them and making them usable for further analysis. As a result, the impact of this new technology is significant, because it allows us to build a complete pipeline from the kernel space to the user space.

Organizations leverage this new technology, because it permits to leverage the combination of safety, performance and observability, to build next-generation security tools that run in a privileged environment.

1.2 Thesis Objectives

This thesis investigates the use of eBPF for security monitoring through the design and implementation of a prototype system that leverages eBPF to collect relevant events from the Linux kernel, transports them to user space, and converts them into structured information for further analysis.

In order to achieve this goal, the thesis will focus on the following specific objectives:

- Design and implement a kernel-side component that uses eBPF to attach to selected kernel hooks and collect raw relevant events from the kernel space.
- Implement a user-space component that receives the raw events from kernel space, decodes their context and arguments, enriches selected fields, and finally outputs them in a structured format that can be used for further analysis.
- Define an event model and decoding pipeline that separates kernel-side data collection from user-space interpretation and presentation.
- Evaluate the prototype in terms of functional correctness, event coverage, and runtime overhead on the target Linux environment.

1.3 Scope of the Work

The work presented in this thesis is a research prototype that investigates the use of eBPF for runtime security monitoring. Its scope is to demonstrate how kernel-level events can be collected, transported to user space, decoded, and exposed in a structured form.

Tracee is used as the main architectural reference, since it represents a production-grade example of eBPF-based runtime security monitoring. Unlike production-grade tools that must support a wide range of kernels and deployment scenarios, this prototype is intentionally tailored to a specific target environment, namely *Rocky Linux 8.10*.

The focus of this work is the implementation of kernel hooks related to process and security events, in particular a selected set of events relevant to process execution, filesystem activity, credential changes, memory operations, namespaces, and kernel-hardening signals.

The resulting architecture is intended to provide a basis for further extensions, including policy-based filtering, anomaly detection, and event correlation.

1.4 Thesis Structure

The remainder of this thesis is organized as follows:

- **Chapter 2** introduces the technical background required to understand the work and discusses related runtime security tools, with particular attention to eBPF architecture, the Linux kernel, the Verifier, the eBPF hooks, and the *Tracee* runtime security tool.
- **Chapter 3** presents the requirements and the design of the proposed system, including the target environment, the main architectural components, and the design choices that shaped the prototype.
- **Chapter 4** describes the implementation of the system, covering the eBPF kernel-side component, the Go user-space runtime, event transport, decoding, filtering, output generation, and the main implementation challenges.
- **Chapter 5** evaluates the prototype in terms of functional behavior,

event coverage, runtime overhead, and limitations observed on the target environment.

- **Chapter 6** concludes the thesis by summarizing the achieved results, discussing the current limitations, and outlining future extensions.

Appendix A

Appendix A

Appendix text here.

Appendix B

Appendix B

Appendix text here.

To my family and friends, for their support.